

Post-Quantum Session Confidentiality

A captured inference session keeps its value for as long as the person who typed the prompt is alive. Morpheus encrypts the answer and sends the question in cleartext.

What the transport does today. Consumer prompts cross :3333 as plain JSON while provider responses are ECIES encrypted. `SessionPromptRequest` takes the provider's public key as an argument and never reads it (`morrpcmessage/mor_rpc.go:471`); `SessionPromptReq.Message` is a `string`, not a ciphertext (`request.go:55`). All four `EncryptString` call sites in `proxy_receiver.go` run provider to consumer. The protected half is the half that usually matters less.

The part that outlives that fix. Every confidentiality primitive in the tree is `secp256k1`: ECIES payloads (`lib/crypto.go`), `morrpc` frame authentication, ECDSA approvals in `SessionRouter`. The ECIES key is the same wallet key handed to `bind.NewKeyedTransactorWithChainID` (`blockchainapi/service.go:1796`), so its public value is recoverable from any Base transaction the account has broadcast. An adversary with a cryptographically relevant quantum computer therefore never attacks the recorded ciphertext; it lifts the static private key off public chain data and reads every session it captured.

Attestation answers a different question. The 5,439-line attestation stack binds TDX `report_data` to the TLS certificate digest of the peer the verifier actually reached, compares the NVIDIA `eat_nonce` in constant time, and pins golden measurements to a Sigstore identity. That is careful work, and it establishes what runs inside an enclave. It establishes nothing about the channel reaching the enclave, and a passive recorder on :3333 never touches the enclave.

Mechanism. Session establishment becomes hybrid: X25519 and ML-KEM-768 combined through HKDF, so the session key survives the failure of either primitive. Traffic keys derive only once the transcript is bound to the on-chain session identity of chain id, diamond, `sessionId`, `bidId`, provider and user, so a channel cannot be transplanted between sessions. Approvals become ECDSA $\wedge$ ML-DSA-65, the lattice leg checked where the approval is already verified once, in the consumer's proxy-router. Base has no ML-DSA precompile and we will not pretend otherwise: settlement stays classical until the chain has one. Solvency has a short horizon and confidentiality an unbounded one. Binding the transcript to the funded session also closes `proxy_receiver.go:525`, where the provider stores `user:<addr> → req.Key`, a key the requester asserts about itself and whose signature is checked against itself (`controller_morrpc.go:125`); the consumer has the same shape at `proxy_sender.go:288`, seventy lines below the on-chain resolution the same file already implements at line 218. Under the handshake the peer key comes from the recovered signer and the `SessionOpened` event, never from the message. Migration is suite-version negotiation, not a fork: the suite identifier sits inside the transcript hash, so a downgrade fails as a bad signature rather than silently. Lux ran this shape on a live mainnet, moving validator authentication onto a hybrid classical and ML-DSA suite with no downtime.

What Morpheus gets

- Prompt confidentiality now, ahead of any post-quantum work.
- Recorded sessions that stay unreadable once a CRQC exists, the property here least amenable to being added later.
- The rebinding path at `proxy_receiver.go:525` closed, peer identity anchored to the address that funded the session.
- A specification implementable without adopting Lux or Hanzo software, with no change to where Morpheus settles.

What we get, stated plainly

- A second production deployment of the Lux post-quantum suite, worth more than a license fee to us, and a relationship with the team whose settlement layer we think inference markets belong on.

First step, and who takes it. Morpheus core lands one node-side diff: anchor the `session.request` signature check to `req.User`, and require `req.Key` to recover to that address. `VerifySignatureAddr` already exists (`mor_rpc.go:625`) and is used on the ping path. No wire change, no new dependency, no contract upgrade, and a patched provider protects its own users whatever the client runs. We will open it with a proof of concept under the existing bounty scope. Everything else rests on that check.

What this does not solve. Nothing here constrains what a provider does with the plaintext it necessarily holds. A sound channel still delivers the prompt to whatever backend that provider configured, and per-second pricing leaves the substitution no on-chain artifact.